Check for updates

Special Issue: Machine Learning for Civil Structures

Structural Health Monitoring
1–20
© The Author(s) 2025
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/14759217251349533
journals.sagepub.com/home/shm

Efficient edge intelligence for onboard data anomaly classification in wireless structural health monitoring using knowledge distillation on low-cost IoT nodes

Xiao Yu[1], Yuqing Zhao[2], Shuaiwen Cui[3], Xinhao He[4], Yuguang Fu[5] and Qun Yang[6]

Abstract

Internet of Things (IoTs) systems with wireless data transmission have been increasingly adopted by structural health monitoring. However, sensor malfunctions and data anomalies have been identified frequently in these systems for long-term deployment, especially in harsh environments. Advances in machine learning (ML) offer promising solutions for effectively detecting, classifying and recovering data anomalies. Most state-of-the-art ML-based solutions rely on powerful workstations or cloud servers, where large amounts of datasets are assumed to be transmitted to a centralized station, leading to the issues of heavy load of data transmission and long delay of subsequent data analytics. To address the issues, this article proposed a novel decentralized approach integrating machine learning with edge computing, aka, edge intelligence, for efficient onboard data anomaly classification. The key innovations lie in the development of an effective ML-based feature extractor adapted to low-cost IoT nodes and an efficient edge computing deployment for ML execution suitable for resource-constrained microcontroller unit-based devices. The proposed solution has been successfully deployed on a newly developed IoT node, *LiftNode*, which is by first time to realize low-cost onboard data anomaly classification in structural health monitoring (SHM) community. Using the dataset collected from a real long-span cable-stayed bridge in China, the proposed solution is found to achieve the accuracy of 96.37% in PC with the model size of 0.29 MB, and comparably, the accuracy of 96.36% in *LiftNode*. The framework and deployment strategies are elaborated, highlighting the potential of edge intelligence for improving SHM systems' responsiveness and resource utilization.

Keywords

Edge intelligence, data anomaly classification, wireless internet of things, structural health monitoring, knowledge distillation

Introduction

The new generation of wireless structural health monitoring (SHM) systems leverages the Internet of Things (IoT) paradigm, integrating smart sensors that not only acquire measurement data but also process and transmit it via the Internet.[1,2] The data collected by edge devices, aka IoT nodes, can be analyzed using machine learning (ML) algorithms to detect patterns or anomalies that may indicate structural weaknesses or the need for maintenance.[3] This enables real-time monitoring of structural integrity by assessing performance, identifying damage or deterioration, and providing a comprehensive understanding of the structure's overall

[1]School of Civil and Environmental Engineering, Nanyang Technological University, Singapore, Singapore
[2]College of Computing and Data Science, Nanyang Technological University, Singapore, Singapore
[3]School of Civil and Environmental Engineering, Nanyang Technological University, Singapore, Singapore
[4]Graduate School of Engineering, Tohoku University, Sendai, Japan
[5]School of Civil and Environmental Engineering, Nanyang Technological University, Singapore, Singapore
[6]Department of Civil and Environmental Engineering, The University of Auckland, Auckland, New Zealand

Corresponding author:
Yuguang Fu, School of Civil and Environmental Engineering, Nanyang Technological University, 50 Nanyang Avenue, Singapore 639798, Singapore.
Email: yuguang.fu@ntu.edu.sg

condition.[4,5] It provides an alternative due to their compact node size, facilitating easy deployment, wireless communication, and their capability for flexible expansion and upgrade,[6] serving as integral components of wireless SHM systems. However, edge devices often face various interferences in the harsh environment, which may result in the transmission of erroneous data. The data quality can be compromised by anomalies such as sensor faults, poor data transmission, harsh environments, unexpected events, and deliberate attacks.[7–10] Due to harsh environment and vulnerability of low-cost hardware, sensors may produce extensive time-series data frequently interspersed with anomalies[11] that represent a considerable obstacle to automated and reliable alerts for potential damage or accidents and can compromise their efficacy for long-term SHM applications.[12] Thus, the timely detection and classification of these data anomalies are vital to safeguarding IoT networks from their detrimental effects.[13] Data anomaly classification in time-series data plays a crucial role in safeguarding the quality of sensor data and diagnosing sensor faults to ensure the accuracy of subsequent decisions and the integrity of the systems.[14] Distinguishing the categories of data anomalies enables engineers to gain deeper insights into the sensor conditions, thereby expediting the maintenance procedures.[15]

Advancements in intelligent sensing and sensor network systems have enabled the acquisition of vast and diverse datasets in SHM. It's impractical to utilize manual identification, which consumes significant human resources and time costs. Subsequently, some previous studies have explored methods to employ statistical methods for analysis. The fundamental principle of the statistical anomaly detection method involves modeling the distribution of "normal" and conducting hypothesis tests grounded in this assumption to divide abnormal as highlighted by Rajasegarar et al.[16] Although traditional manual identification and statistical methods have been extensively employed in SHM, they often rely on expert experience or are constrained by specific assumptions and conditions. These traditional approaches may not be effective and scalable to handle large-scale and complex structural data, leading to accuracy and real-time performance challenges. Moreover, when real-world datasets diverge from these assumed distributions, it results in a model mismatch, further exacerbating the limitations of these conventional methods.[17]

With the advancements in ML and artificial intelligence (AI) technologies, data-driven approaches have emerged as effective means to enhance the performance of SHM systems, eliminating the need for a physical system model.[18] These sophisticated methods can automatically extract features, identify complex patterns, and enable real-time monitoring, thereby significantly improving the accuracy and efficiency of monitoring. ML methods for data anomaly detection can be mainly classified into unsupervised and supervised learning. Unsupervised ML methods for data anomaly classifications commonly applied in SHM include principal component analysis, k-means clustering, hierarchical clustering[19] and autoencoder-based method,[20] which often oversimplify data anomaly detection to binary classification, thereby losing the specificity of anomalous classes. The supervised ML counterparts herein include the random forests (RFs) classifier,[21,22] support vector machines (SVMs), and decision tree framework[23] to do the data anomaly detection. However, traditional ML methods often suffer from insufficient accuracy, which has led to the emergence of deep learning techniques for their promising performance. Deep learning (DL) encompasses methods such as convolutional neural networks (CNN),[24–28] deep autoencoders (DAEs),[27] recurrent neural networks (RNNs),[28] generative adversarial networks (GANs).[29] Deep learning methods typically require the extraction of data features, such as statistical features,[24] two-dimensional grayscale images,[22,30] time–frequency spectrograms,[15,25] and RGB image representations,[19] before training.

While state-of-the-art deep learning-based strategies offer significantly higher accuracy, most of them are processed in a centralized manner, that is, collecting data from all the sensor nodes and processing it in a central station (e.g., workstations or cloud servers). Central processing not only increases the transmission load for wireless SHM systems, which may result in data loss issues but also extends the significant delay of subsequent data analytics for timely structural condition assessments.[31] The unnecessary storage and transmission of anomalous sensor data can also result in severe channel interference and lead to inefficient utilization of energy and bandwidth.[26,27] Some practices considered communication overhead[32,33] and explored hierarchical structure.[34] The hierarchical structure was established by allocating each group of low-capacity sensors to one of the few more powerful sensors according to their spatial proximity,[35] which only partially addresses the challenge. To fully address it, it is preferable to delegate these tasks to the "edge" to optimize resource utilization and enhance responsiveness, which mainly consists of two subchallenges: (1) minimizing the deep learning model size while maintaining the accuracy, and (2) compiling and deploying on edge devices which usually use alternative resource-efficient language and architectures. Wang et al.[36] integrated the AI algorithm into NVIDIA Xavier NX edge computing boards to detect data anomalies caused by sensor faults before large datasets are uploaded to the

cloud platform. Kosmas et al.[37] investigated distributed adaptive diagnosis methods, focusing on fault detection, isolation, and accommodation, but the ANN is deployed on a portable main station, essentially a small computer, rather than achieving true edge deployment. However, most of the existing studies adopt single board computers (SBC) as edge devices which actually bypass the two aforementioned subchallenges to some extent, because they have sufficient resources for deep learning and support Python language for seamlessly AI programming. These SBC devices generally require wall power and are more expensive than microcontroller unit (MCU)-based devices; hence, they are not applicable for long-term large-scale wireless SHM applications. In sum, to the best knowledge of the authors, none of the existing studies have successfully implemented and achieved promising results in onboard data anomaly classification using low-cost IoT nodes with constrained resources.

This article fully addresses the two subchallenges mentioned in the previous paragraphs for data anomaly classification by contributing two innovations: (1) developing a novel effective encoder-enhanced classification model (EECM) feature extractor and a lightweight classifier, which uses knowledge distillation (KD) to transfer knowledge from a robust teacher model to significantly minimize the ML model size while keeping high accuracy; and (2) establishing an edge-computing deployment framework to successfully compile, download, and execute the ML model on low-cost IoT nodes which run MCUs. This approach targets nonperiodic signals (e.g., vibration) and has fully realized the efficient edge intelligence for wireless structural health monitoring systems. Compared to the existing studies built upon expensive and power-hunger SBC-based IoT nodes, this study uses low-cost MCU-based nodes, making it suitable for cost-effective densely deployment of large-scale civil infrastructure. The subsequent sections are structured as follows: section "Development of an effective ML-based feature extractor and classifier" outlines the proposed methodology for training the EECM and the knowledge distillation process used to transfer knowledge to train a distilled lightweight classification model (DLCM) effectively. In section "Edge intelligence framework for IoT-based wireless SHM applications," the coding part and the theory of deployment on sensor nodes are presented. In section "Case study and result discussion," validation and results on the real-time data from bridges are illustrated. Finally, the concluding remarks are provided for data anomaly classification and decentralized deployment framework.

Development of an effective ML-based feature extractor and classifier

In this section, the strategy of minimizing ML model size while maintaining classification accuracy is presented, in order to make the model suitable for edge intelligence on resource-constrained MCU-based IoT nodes. Many ML and DL models for data classification often demonstrate a positive correlation between model complexity and task performance. Although such complex models excel in classification tasks, they demand significant computational resources, rendering them unsuitable for deployment on edge devices. To address this challenge, the proposed approach involves training an EECM to be capable of robust anomaly classification as a teacher model and applying knowledge distillation to produce a smaller, resource-efficient student model, DLCM, as shown in Figure 1. The distilled model maintains comparable accuracy while being optimized for deployment on MCU-based IoT nodes.

Development of an EECM

As shown in Figure 2, an EECM is developed for effective data anomaly classifications, which are designed to extract, compress, and classify features. First, statistical and histogram features are extracted from the raw time series data. Then, an encoder based on the attention mechanism is applied to quantify the correlation between these features, thereby highlighting information crucial for the subsequent classification task. Finally, fully connected linear layers are used to classify different data types. This section mainly describes the strategy about how to effectively extract features from raw data using domain knowledge and a transformer encoder.

Feature extraction from raw time-series data. The efficacy of data-driven methods is intricately tied to the quality of feature extraction.[38] Directly analyzing time series data without explicit feature extraction offers convenience and reduces the time and effort required for data preprocessing but requires more complex and deeper neural networks that are computationally intensive and time-consuming.[39,40] This reliance on raw time-domain signals for data anomaly classification can lead to misinterpretations due to similar anomalous data patterns.[41] The uncertainty in raw data complicates classification tasks, increasing network complexity and extending computational time and power requirements. Additionally, the large volume of original data points further impacts computational efficiency.

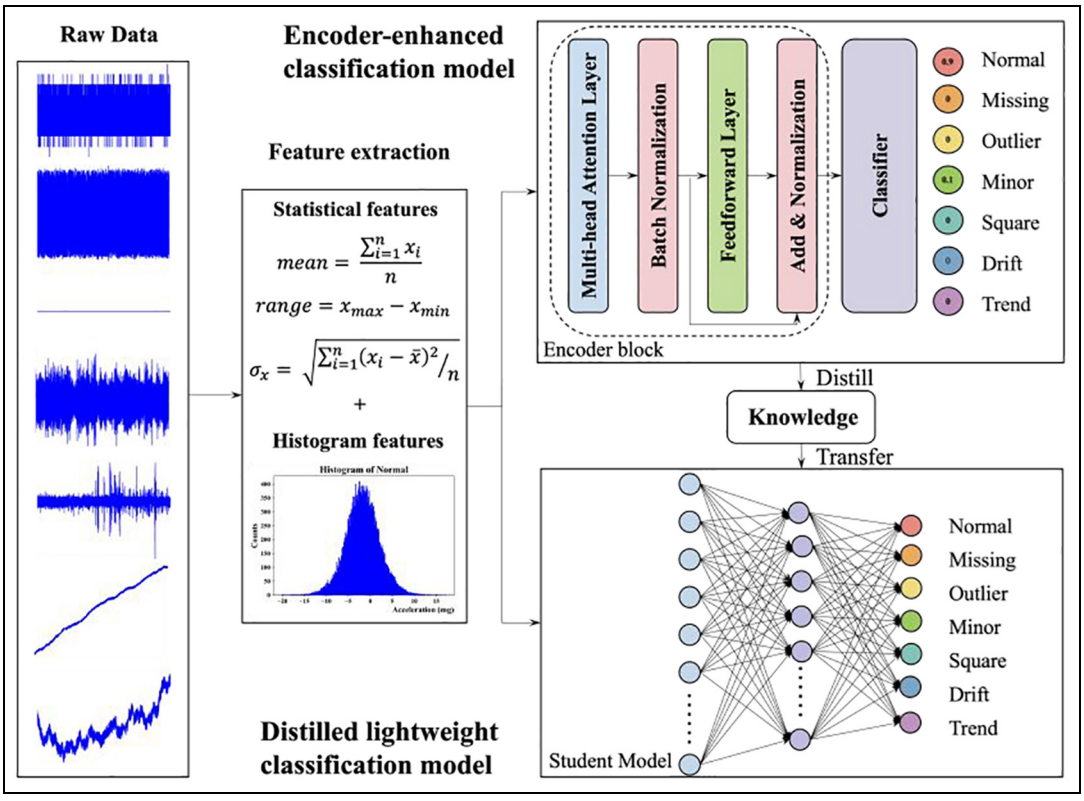


Figure 1. The framework of the model development.

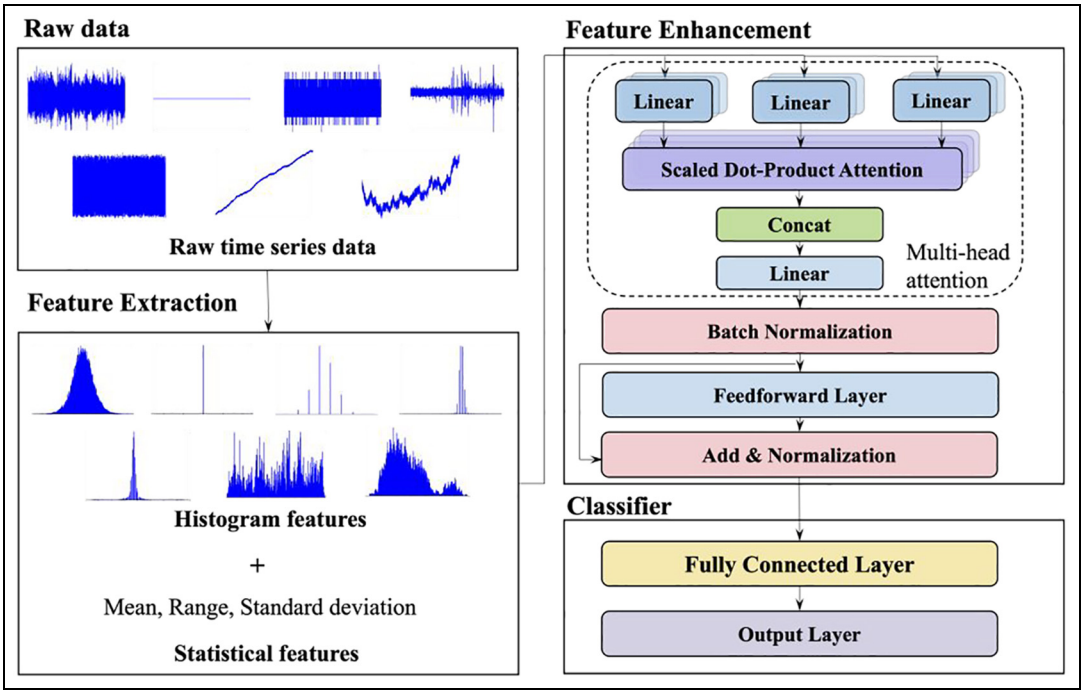


Figure 2. The structure of the encoder-enhanced classification model.

Therefore, three statistical features (mean, range, and standard deviation) and histogram features are applied to process raw time series data as signal features that could identify the fault type. Extracting features from raw data is a data compression process that serves not only to conserve storage capacity but also significantly enhances the computational efficiency of the training algorithm.[11] The statistical features for a sample x are expressed as follows:

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n} \tag{1}$$

$$r_s = x_{\max} - x_{\min} \tag{2}$$

$$\sigma_x = \sqrt{\sum_{i=1}^{n} (x_i - \bar{x})^2 / n} \tag{3}$$

where $\bar{x}$ denotes the mean value of the signal sequence, x_i denotes the $i$thth data point of the signal sequence, n denotes the length of the signal sequence, r_s denotes the range of the signal sequence, $x_{\max}$, and $x_{\min}$ denote the maximum and minimum values of the signal sequence, respectively, σ_x denotes the standard deviation of the signal sequence. The mean value is calculated across the time series data to capture the central tendency. Range values are derived by determining the difference between the maximum and minimum values along the same dimension, offering insights into the variability of the time series data. The standard deviation values are computed to quantify the dispersion of data points from the mean, thereby characterizing the dataset's consistency or uniformity. In addition, histogram features are extracted to capture the distributional properties of the signal. Each sequence is divided into equal-width bins over the range from the minimum to the maximum value, and the number of values falling within each bin is counted. The histogram features provide coarse-grained information about the shape and amplitude spread of the signal, complementing the statistical features and improving anomaly classification performance. Feature dimensions are initially set as powers of two (e.g., 64, 128, 256) to improve memory alignment and computational efficiency on resource-constrained microcontrollers. These binary-compatible sizes simplify buffer allocation and speed up matrix operations during inference. The final feature dimension is selected through model-specific fine-tuning based on performance metrics, ensuring the best trade-off between accuracy and computational cost.

Feature expression enhancement with a transformer encoder. Extracting statistical and histogram features reduces data length and simplifies model complexity. However, relying solely on these features limits the accuracy and precision of data anomaly classification, as they may not fully capture the intricate patterns in the data. To address this limitation, a transformer encoder known for its ability is employed to identify complex patterns and dependencies in sequential data.[42] The encoder processes input features through attention and feed-forward layers, dynamically assigning attention weights and applying nonlinear transformations to improve the discriminative power of the extracted features. This approach enables the model to learn and encode complex relationships within the data, leading to more effective anomaly classification. Additionally, residual connections and batch normalization are integrated to improve stability and performance.

(1) Multihead attention mechanism

Self-attention evaluates the significance of each feature relative to others within a sequence, while multihead attention allows the model to focus on multiple parts of the input simultaneously. The keys, values, and queries are derived from the same output of the preceding feature extraction layer, allowing each position in the sequence to attend to all others. This mechanism captures feature dependencies, enhancing the effectiveness of feature extraction. Self-attention specifically enables the model to establish connections between different positions within a single sequence, facilitating the computation of a comprehensive representation of that sequence. The attention function is as follows:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{4}$$

where Q denotes the query of the signal, K denotes the key of the signal, V denotes the value of the signal, K^T denotes the transpose of the key, and $\sqrt{d_k}$ denotes the square root of the dimension of key. The attention function maps a query and a set of key-value pairs to an output which is a weighted sum of the values. The weights are calculated based on the compatibility between the query q, and each key k.[43] The attention function processes a batch of queries as a matrix Q, with keys and values organized into matrices K and V, respectively. The output matrix is obtained by calculating the dot products between the queries and keys, normalizing by $\sqrt{d_k}$, and applying a Softmax function to derive the weights.

A single attention head, due to its averaging nature, may limit the model's ability to focus on multiple aspects of the data simultaneously, potentially constraining its capacity to capture complex relationships and dependencies. Multihead attention addresses this

limitation by enabling the model to focus on different representation subspaces and positions concurrently, thereby enhancing information processing efficiency, as given in Equation (5). The approach utilizes h-learned linear projections that transform queries, keys, and values to dimensions d_k, d_k, and d_v, respectively. For each head, the dimensions are set as $d_k = d_v = d_{\text{model}}/h$, allowing the attention function to be applied in parallel to the transformed queries, keys, and values. The resulting output values are then concatenated and passed through another linear projection to produce the final values. This method maintains a computational expense comparable to that of single-head attention with full dimensionality.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head1}, \ldots, \text{head}_h)$$
$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (5)$$

where the projections are parameter matrices $W_i^Q E R^{d_{\text{model}} \times d_k}$, $W_i^K E R^{d_{\text{model}} \times d_k}$, $W_i^V E R^{d_{\text{model}} \times d_v}$, and $W^O E R^{hd_v \times d_{\text{model}}}$. The multihead attention mechanism enhances the model's ability to capture a broader range of information and relationships within the data, thereby improving its understanding and representation of complex patterns. This approach not only accelerates data analysis but also efficiently allocates resources to extract the most relevant information. By assigning weights and computing interfeature correlations, the attention mechanism boosts the representational power of the extracted features, making the model more effective in processing and interpreting intricate data.

(2) Feedforward layer

The other sublayer of the encoder applies a positionwise feedforward neural network to each position in the sequence independently, as given in Equation (6). The feedforward layer processes the output of the attention layer by applying two linear transformations and a ReLU activation function in between, thereby generating a refined feature representation:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (6)$$

The first linear transformation increases the dimensionality of the features, allowing the network to capture more complex patterns and interactions within the data. The ReLU activation function introduces nonlinearity, enabling the model to learn and represent intricate relationships that cannot be captured by linear transformations alone. The second linear transformation reduces the dimensionality back to the original size, ensuring that the overall structure of the data is preserved. By incorporating these transformations, the feedforward layer generates a more refined feature representation, enhancing the encoder's ability to model and understand the underlying structure of the input sequence.

(3) Batch normalization and residual connection

Batch normalization is applied after each sublayer in the encoder, with residual connections around each sublayer to stabilize the training process. Unlike the conventional transformer encoder, which uses layer normalization, batch normalization is opted for because each feature extracted from the time series data in the study are independent. Batch normalization scales feature independently within each batch, whereas layer normalization normalizes the output of each layer. This modification has demonstrated improved performance, indicating that batch normalization is an effective optimization strategy for handling independent features in the specific application. Residual connections help maintain gradient flow and mitigate the vanishing gradient problem, thereby enhancing the model's stability and convergence. Since the sensor-collected data are sequential and lack cross-data type dependencies, embedding or positional encoding in the transformer encoder is unnecessary.

Lightweight data anomaly classifier. After feature extraction and compression through the transformer encoder, the resulting sequence of feature vectors is considerably shorter yet richer in discriminative information compared to the original signal. Subsequently, this enhanced sequence of feature vectors will be input into a neural network for classification in the subsequent step. The data anomaly classifier includes a flattened layer, two fully connected layers of 64 neurons and 32 neurons, respectively, and an output layer of seven neurons same as the number of data classes. The feature vector ranges from -1 to 1 after normalization, the Tanh activation function is used in the fully connected layer establishing mappings between features and output. The output layer applied the softmax activation function, with the element representing the probability of belonging to each class. The supervised learning approach is applied utilizing a well-suited classification algorithm. The classifier is trained on a labeled dataset comprising both normal and anomalous instances, allowing it to learn the distinguishing characteristics of anomalies and differentiate them from regular patterns. Mean-square error is used as the objective function to measure the difference in probability distribution between the true and predicted classes.

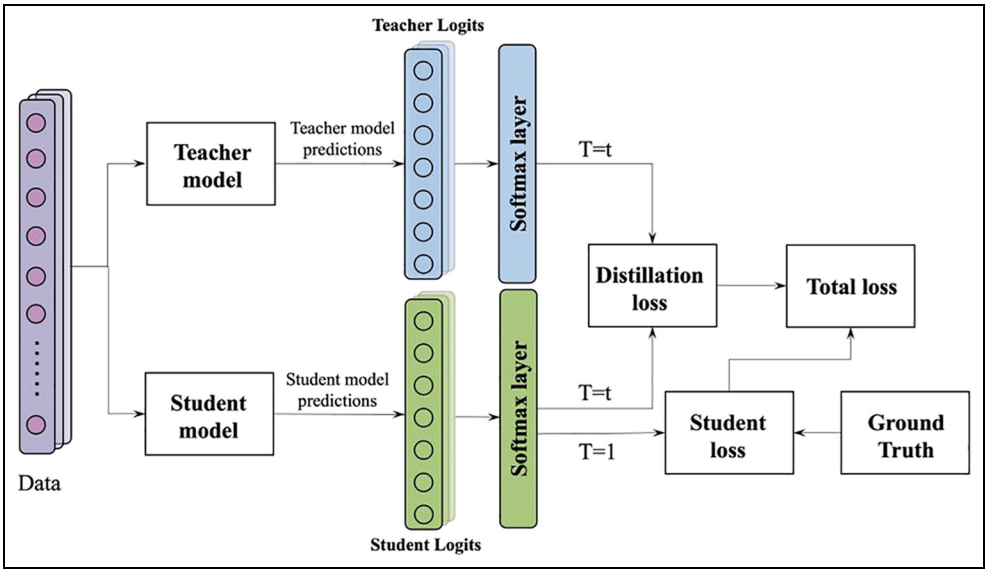


Figure 3. Knowledge distillation process.

Machine learning model compression using knowledge distillation

The EECM demonstrates excellent performance in accurately classifying data anomalies. However, its computationally intensive nature makes it not suitable for deployment on edge devices. To address this limitation, knowledge distillation[44] is employed to reduce model size by transferring knowledge from a high-capacity teacher model to a compact and lightweight student model. The knowledge distillation process uses response-based knowledge and leverages both hard labels (ground truth) and soft labels (predictions from the teacher model) to guide the training of the student model DLCM. This dual-label approach ensures that the student model captures not only the ground truth but also the correlation among classes of the teacher model. The teacher model EECM is pre-trained using a transformer encoder, which extracts and compresses features from raw time-series data, yielding high-quality feature representations for superior data anomaly classification. During distillation, the teacher model outputs logits, which represent the probability distribution over classes. These logits are then transformed into soft targets using a softmax function with a temperature parameter T, as illustrated in Figure 3. The temperature T smooths the probability distribution, highlighting interclass relationships that guide the student model DLCM during training.

Knowledge distillation relies on various forms of knowledge transfer, including response-based knowledge, which refers to the neural outputs of the teacher model's final layer. The primary goal is to mimic the teacher model's predictions directly. This approach is simple yet effective for compressing large models and has been widely used in various applications. The response-based method captures "dark knowledge" embedded in the teacher model's soft targets, which contain critical insights about the relationships between different classes.[45] The soft targets are computed as Equation (7):

$$p(z_i, T) = \exp(z_i/T)/\sum_j \exp(z_j/T) \tag{7}$$

where z_i is the logit for the i_{th} class, and T represents a temperature factor.

The training process employs a combined loss function to balance the influence of hard and soft targets, and the student model is trained to minimize the total loss, as defined in Equation (8). Hard labels are the true class labels used during training, providing ground-truth information for the classification task. These ensure that the student model maintains accuracy in predicting the correct class labels. Soft labels are from the teacher model that help the student model to learn the intricate patterns and interclass relationships captured by the teacher model, effectively mimicking its behavior. Hard labels provide the correct class for each sample, while soft labels represent the probability distribution predicted by the teacher model. This dual-label approach allows the student model to learn the underlying decision boundaries and the relations between different classes from the soft targets of the teacher model's predictions:

$$\mathcal{L}_{\text{total}} = (1-\alpha)\mathcal{L}_s + \alpha\mathcal{L}_{KD} \tag{8}$$

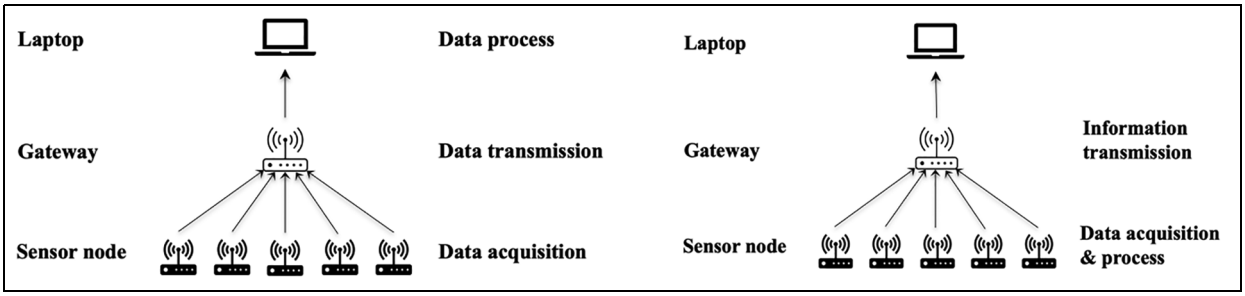


Figure 4. Centralized (left) and decentralized (right) data processing schemes.

where $\mathcal{L}_s$ is the classification loss between the predictions of the student model and the hard labels, and is the knowledge distillation loss between the predictions of the teacher and student model, α is the hyperparameter that adjusts the term's contribution to the final loss. Student model classification loss applied cross-entropy loss that measures the difference between the predicted probabilities of the student model and the ground truth, ensuring that the student model retains accurate class prediction capabilities. Knowledge distillation loss applied Kullback–Leibler (KL) Divergence which quantifies the difference between the probability distributions (soft labels) generated by the teacher and student models. The KL divergence facilitates the transfer of nuanced interclass relationships from the teacher to the student model, enabling the student to mimic the teacher's behavior effectively. The hyperparameter α controls the trade-off between the contributions of two losses, allowing the model to balance the learning of hard and soft labels. The distillation loss for response-based knowledge can be formulated as Equation (9):

$$\mathcal{L}_{KD} = \mathcal{L}_R(z_t, z_s) \tag{9}$$

where $\mathcal{L}_R(.)$ indicates the divergence loss of logits, and z_t and z_s are output logits of teacher and student, respectively. Knowledge distillation ensures that the student model achieves a comparable level of accuracy while being computationally efficient and deployable on resource-constrained MCU-based IoT nodes. The resulting student model achieves a balance between computational efficiency and predictive accuracy, making it highly suitable for deployment on resource-constrained edge devices such as MCU-based IoT nodes. This lightweight model maintains a comparable level of accuracy to the teacher model while being significantly smaller and more efficient.

Edge intelligence framework for IoT-based wireless SHM applications

This section presents the development of an edge intelligence framework, which enables the efficient execution of lightweight machine learning models, including ML for data anomaly classification, on low-cost IoT nodes equipped with MCUs. The framework facilitates the translation of program language, the deployment of ML models onto MCUs, and the execution of classification tasks directly on edge devices. By optimizing resource usage and ensuring robust performance in decentralized environments, this approach significantly enhances the application of edge intelligence in wireless structural health monitoring systems. This section presents the definition and benefits of decentralized data processing, the description and advantages of MCU-based IoT nodes, and a deployment framework for edge intelligence which are built upon decentralized data processing and MCU-based IoT nodes.

Centralized and decentralized data processing scheme for wireless SHM systems

Many researchers have developed ML/DL models with high accuracy and powerful capabilities in various SHM applications; however, most of these models are designed for centralized data processing. In particular, due to their large size and computational demands, they are only applicable for centralized data processing schemes, where raw data from multiple sensors is aggregated by a gateway and then processed by a workstation or cloud server (Figure 4). This method leverages powerful processing capabilities and sufficient storage in workstations or cloud servers to execute ML/DL applications but may introduce long latency, lead to severe data inundation, and also create a single point of failure.[31] In contrast, a decentralized data processing scheme distributes processing tasks across multiple edge devices (e.g., IoT nodes), each of which is equipped with its own sensors and computational resources,[46] and only computing results/information is transmitted back to the users wirelessly. Accordingly, it reduces latency significantly by processing data locally, enhances system robustness by eliminating single points of failure, and improves scalability.

However, deploying algorithms, especially ML models in this study, directly on IoT nodes for edge computing is challenging due to limited computing capabilities and small memory resources in each IoT node. These limitations prevent the deployment and execution of complex algorithms, for example, ML/DL models, on IoT nodes, necessitating the efficient utilization of available resources to balance performance and functionality. Note that, there is another decentralized data processing scheme, where a network of sensor nodes is grouped into several clusters. In each cluster, one node acts as cluster head, and the remaining as leaf nodes. The leaf nodes still transmit raw data wirelessly to the cluster head and the cluster head processes the aggregated data locally before transmitting the result to a gateway or cloud server. This is more complex and advanced, which is not in the scope of this study.

Edge computing, a buzz word recently, is a decentralized data processing scheme that brings data processing closer to the sensors that generate it,[47] enhancing efficiency and scalability for rapid data analytics and decision-making.[48] Instead of relying on the cloud or centralized data centers, edge computing can transform raw data into valuable information by enabling data processing and analysis at the network edge, thereby reducing the workload for wireless sensor networks and mitigating the challenges posed by data inundation.[49] By handling data processing on board, edge computing minimizes network latency and enhances the performance of wireless sensor nodes.[4,31] However, wireless smart sensors with edge computing capabilities face significant challenges due to constraints in memory capacity, power supply, and microprocessor speed, limiting the complexity of applications and the number of algorithms that can be implemented at the hardware level.[50] Though several researchers have developed successful edge computing applications for wireless SHM applications, limited studies have been found to realize edge computing of machine learning models, aka edge intelligence, in the SHM community. This section will explore to realize the edge intelligence on a low-cost MCU-based IoT node.

LiftNode: a low-cost wireless IoT node

This subsection will present the hardware setup for the edge intelligence framework in this study, aka, a low-cost wireless IoT node recently developed by Laboratory for Intelligent Infrastructure Technology, at Nanyang Technology University. Before showing detailed information of the IoT node, it is worthwhile to clarify the choice of MCU-based nodes in this study. One of the most crucial components of edge nodes is the onboard central processing unit,[50] which enables intelligent capabilities including real-time data processing, advanced analytics, and edge computing. A variety of devices utilize the microprocessor unit (MPU), which is a high-performance processor core with a large cache and robust computing and processing capabilities. These MPUs are commonly found in SBCs, for example, Raspberry Pi and Jetson, which leverage their processing capabilities to handle demanding computational tasks efficiently. MPUs are always applied in city-scale devices such as intelligent furniture, automated driving, and traffic management systems, delivering strong performance. However, they require substantial power, making them feasible only in applications where constant power sources are available. In contrast, MCUs have higher integration, lower power consumption, and reduced design costs, making them suitable for embedded systems that operate without a constant power supply, such as battery-operated or energy-harvesting devices. This makes MCUs particularly suitable for large-scale outdoor structural deployments, where consistent power access is often challenging, and energy efficiency is crucial for long-term operation. MCU combines a processor core, memory, and peripheral interfaces on a single chip, enabling efficient control of electronic devices. They are optimized for real-time applications, providing low power consumption, high reliability, and cost-effectiveness. However, the trade-off for these advantages is the limited onboard resources.

This research leverages *LiftNode*, a newly developed low-cost sensor built upon MCU, to deploy the data anomaly classification model and enable edge computing of machine learning models. The *LiftNode's* hardware and software components are reviewed to illustrate its capabilities and implementation. *LiftNode* adopts a "main control board + peripherals" architecture, making it ideal for a broad spectrum of applications. The main control board as shown in Figure 5, applies an STM32 computing module, which utilizes an ARM Cortex-M MCU (up to 480 MHz) known for its versatility, stability, high performance, and low power consumption. It's equipped with a comprehensive memory architecture including 1 MB of built-in SRAM and 32 MB of SDRAM for dynamic data handling, complemented by multiple ROM options, including 2 MB of built-in FLASH, 8 MB of QSPI FLASH. Besides, the board supports a TF card of up to 32 GB, providing ample storage for data logging. With dozens of I/O pins, USB type-C connectivity, and essential debugging interfaces, *LiftNode* is well equipped for various SHM applications. The peripheral part is under active development and it adopts a modular design, meaning it can be tailored to specific applications. According to respective uses, the peripherals can be categorized into three types: sensing, communication, and interfacing. For sensing, it covers

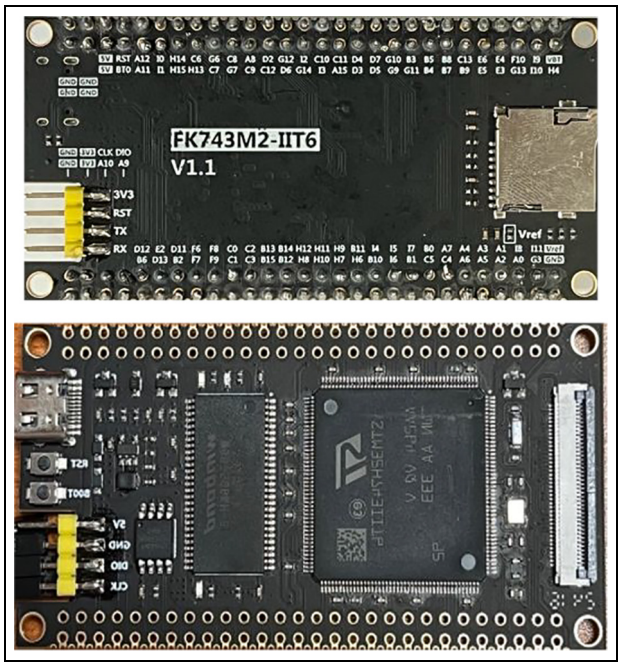


Figure 5. The main control board of the *LiftNode*.

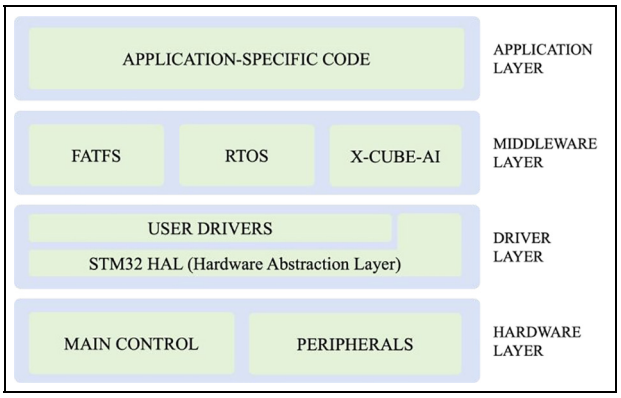


Figure 6. The framework of the *LiftNode* IoT node.
IoT: Internet of Thing.

accelerometers (ADXL362 and ADXL355), environmental sensors (DHT11), and camera (OV5640), enabling comprehensive data collection across different parameters; for communication, it includes Wi-Fi ESP8266, Bluetooth HC-05, and cellular SIM-7600, providing support for wide range of IoT uses; for interfacing, it features an OLED screen to facilitate instant and convenient result displaying.

The *LiftNode* software architecture is designed to leverage the full capabilities of its hardware while maintaining flexibility for application-specific requirements. The system is structured into multiple layers, as shown in Figure 6, of which the bottom layer stands for the hardware, including the main control and various peripherals. Inside the driver layer, the STM32 HAL (Hardware Abstraction Layer) library is used as a foundation for user-customized drivers, providing unified, reliable, smooth, and convenient control to lower-level hardware. Middleware includes a real-time operating system (RTOS) for task management, FATFS for file system handling, and X-CUBE-AI for on-device AI capabilities. The application layer is reserved for custom application-specific code, allowing users to tailor the system to their needs. Key features of the software framework include edge computing capabilities, such as Cortex Microcontroller Software Interface Standard – Digital Signal Processing (CMSIS-DSP) for digital signal processing and Cortex Microcontroller Software Interface Standard – Neural Network (CMSIS-NN) for neural network inference, making *LiftNode* a powerful platform for real-time data processing and intelligent decision-making in edge environments.

Edge computing deployment framework for onboard machine learning applications

Deploying machine learning models on microcontrollers poses unique challenges due to limited computational resources and memory constraints. The combination of robust hardware and supportive software infrastructure of *LiftNode* positions it as a preferred choice for implementing advanced machine learning models, such as ML classifiers, in embedded systems. The STM32 MCU used in *LiftNode* is part of a robust ecosystem supported by development tools and comprehensive software libraries. This ecosystem significantly enhances the efficiency of developing and deploying embedded applications. In this study, several free and open-source software tools are utilized, including STM32CubeMX, STM32CubeIDE, and X-CUBE-AI, all of which are available under open-source licenses. These tools and libraries simplify tasks such as peripheral configuration, code writing and debugging, microcontroller programming, and firmware management. Their free availability and open-source nature facilitated the development process and ensured accessibility for replication and further research.

STM32 microcontrollers, equipped with the STM32Cube.AI framework, offer a viable solution for

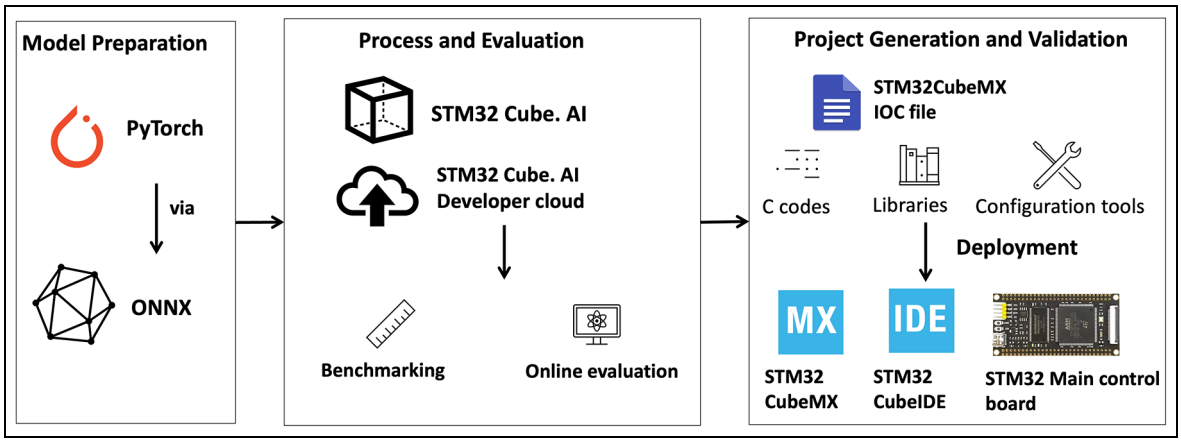


Figure 7. The framework of deployment for edge intelligence.

such deployments. The deployment process involved utilizing the STM32Cube.AI and the Developer Cloud to process the custom classification model and realize the real implementation on board. The deployment framework is shown in Figure 7. The STM32Cube.AI framework provides quantization and optimization options for code generation tailored to MCU targets, allowing further optimization of the model to suit specific STM32 microcontrollers. The STM32 cube developer cloud is an online platform to load models and simulate the execution of ML models on different MCU boards. The network will be executed on an STM32 board farm hosted by ST using random inputs to measure inference times during benchmarking. The complete project including all necessary C code, libraries, and configuration files will be generated to run the model on the target. The STM32CubeMX offers a graphical configuration tool for peripheral setup and initialization code generation and STM32CubeIDE provides an integrated environment for coding, debugging, and project management.

The detailed steps for implementation involve model preparation and upload, process and evaluation, and project generation and deployment to validate on simulated board and real board, each step being crucial for a successful deployment of ML models on STM32 microcontrollers.

(1) Model preparation and upload:

The initial step involves converting the trained model from its original framework format (e.g., PyTorch) to the ONNX format. This conversion ensures that the model is in a hardware-agnostic format that can be processed by the STM32Cube.AI framework. The ONNX model is then uploaded to the STM32 Developer Cloud, allowing users to simulate and benchmark models on various STM32 microcontrollers hardware.

(2) Process and evaluation

Once uploaded, the model undergoes benchmarking and online evaluation. Benchmarking on STM32Cube.AI involves evaluating the performance of machine learning models on STM32 microcontrollers in terms of memory usage, execution time, and power consumption. The optimized model is executed on an STM32 board farm hosted by STMicroelectronics and simulated with random inputs to obtain inference times and benchmark the model's performance. The results from the benchmarking phase are listed and compared to select the most suitable STM32 microcontroller for deployment based on performance metrics.

(3) Project generation and validation

Once the optimal microcontroller is selected, STM32Cube.AI generates a complete project. This project includes the necessary C code, libraries, and configuration files to run the model on the target hardware. The generated code is imported into STM32CubeMX and applies X-cube AI to integrate artificial intelligence into STM32 microcontrollers. STM32CubeMX ensures that the model code is correctly integrated with the rest of the microcontroller's firmware. The integrated project is simulated in a desktop environment to verify the accuracy of the model before deploying the optimized code onto the physical board for real-world testing. The input format for X-Cube-AI is (batch, height, width, channel), and the model is configured to align the feature dimension with

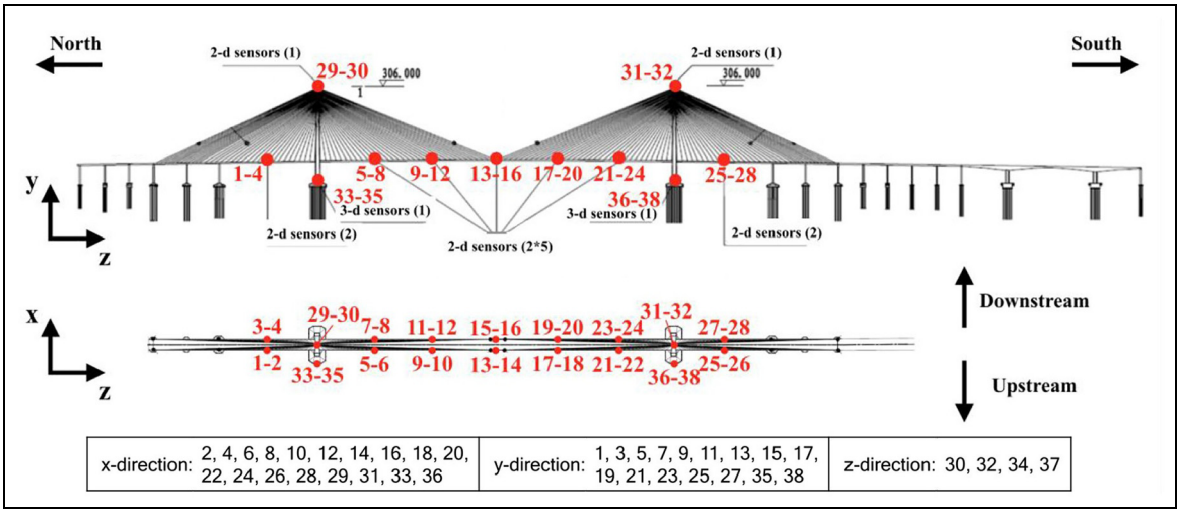


Figure 8. The bridge and placement of accelerometers on the deck and towers.

the width dimension to ensure compatibility with the platform's input requirements. The model's accuracy is assessed by implementing it in C and running predictions against known datasets, which ensures the model's inference capabilities are tested thoroughly. The simulation and testing phase are crucial for identifying and addressing any potential issues before deployment. Finally, the optimized code is flashed onto the physical STM32 microcontroller using a debugger/programmer interface supported by STM32CubeIDE. Finally, the model is tested on the computing board of *LiftNode* under actual operating conditions to ensure it performs as expected in the real world.

In sum, the proposed edge intelligence framework shows a systematic approach integrating advanced machine learning models into embedded systems, ensuring efficient performance and good accuracy while leveraging the robust development tools and ecosystem provided by STMicroelectronics. It is adaptable to a broad range of lightweight machine learning models, including fully connected networks, decision trees, and shallow convolutional architectures. The input shape is required in the form of (batch size, height, width), and the model needs to be convertible to ONNX format and structurally supported by STM32Cube AI. It is limited in handling models with large memory footprints and doesn't support dynamic tensor shapes. Despite these constraints, the framework remains highly suitable for edge applications in structural health monitoring.

Case study and result discussion

The proposed data anomaly classification model is validated using SHM data collected from field tests on a bridge in China. The bridge and its dataset are first briefly described, and then, the validation tests and results are discussed. Finally, a comparison between different methods is presented to demonstrate the efficiency of the proposed solution.

Dataset description

Dataset collection. The proposed method is validated using a dataset from a long-span cable-stayed bridge in China, provided by the IPC-SHM community.[12] This dataset includes acceleration data collected over 1 month (31 days) from 38 strategically positioned accelerometers on the bridge, as illustrated in Figure 8. The data are organized into a 1-h-long time series, resulting in 28,272 instances ("38 × 31 × 24"). Each accelerometer operates at a sampling frequency of 20 Hz, producing 72,000 data points per time series ("3600 × 20"). The dataset is classified into seven categories: normal data and six types of anomalies—missing, minor, outlier, square, trend, and drift, as shown in Figure 9. Normal represents typical oscillation time response and peak-like frequency responses. Missing denotes a complete or substantial absence of response data, resulting in null or constant of time and frequency responses. Minor indicates responses with significantly reduced amplitudes compared to normal sensor data. Outlier identifies responses with one or more data points deviating significantly from the expected distribution. The square represents responses resembling square waves. Trend demonstrates discernible trends in the time domain, accompanied by peak values in the frequency domain. Lastly, drift signifies nonstationary vibration patterns with random drift.

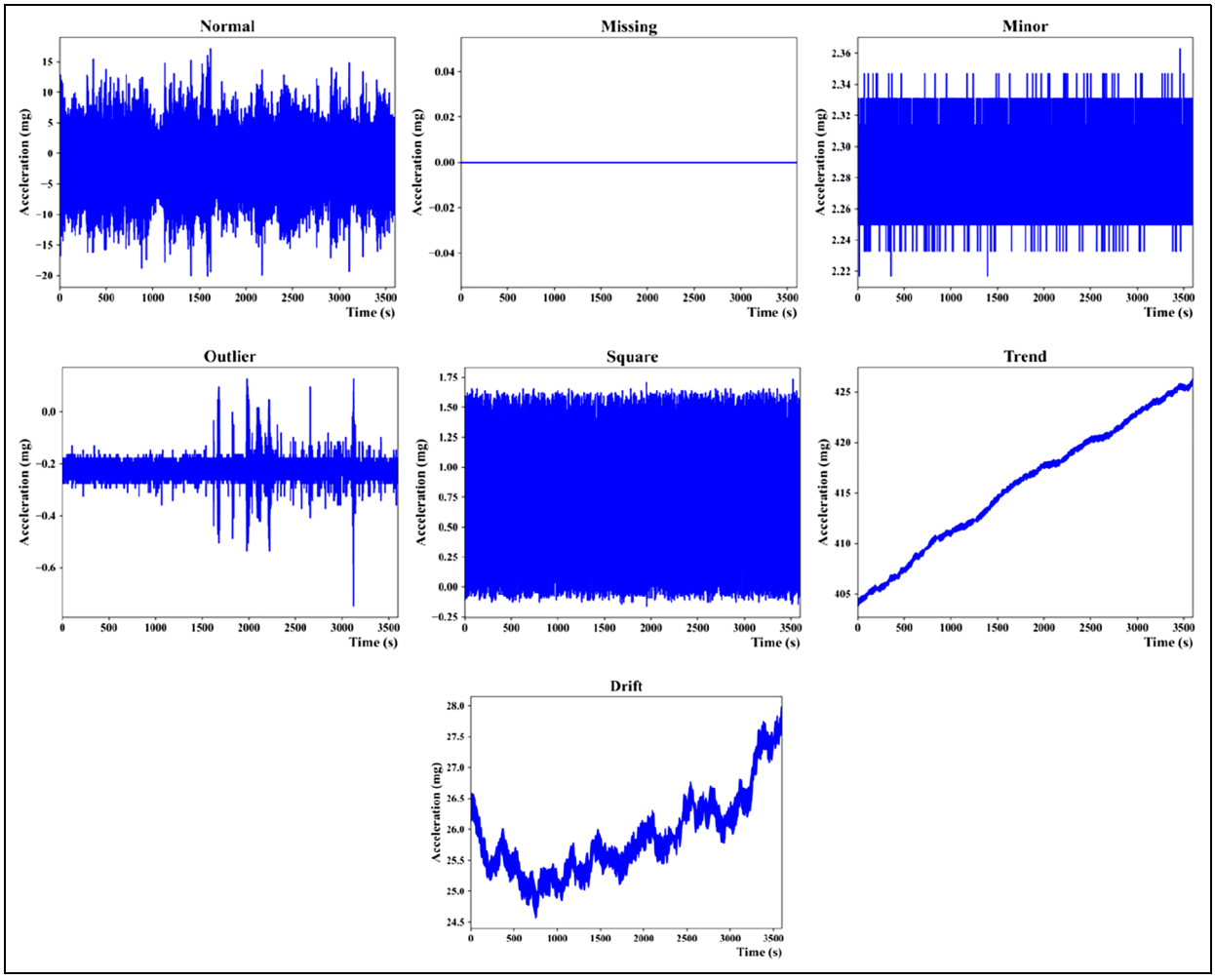


Figure 9. Time-series representation of each data type.

Dataset balance. Sensors typically operate under normal conditions, with anomalies being rare, short-term events in the SHM system.[42] This leads to an imbalance in the dataset, where the "normal" class contains approximately 12,000 samples, while the "outlier" class has only 523 samples. Such imbalance risks the network learning skewed features, resulting in biased predictions, overfitting, or underfitting, and ultimately compromising the model's classification accuracy with erroneous predictions, including false positives and negatives.[11] To mitigate the challenges posed by class imbalance in machine learning, various strategies have been proposed and employed. Over-sampling techniques aim to augment the representation of minority classes by either duplicating existing samples or generating synthetic samples using methods such as synthetic minority over-sampling technique (SMOTE)[11] and adaptive synthetic sampling (ADASYN). Conversely, under-sampling methods reduce the number of samples in the majority class to balance the dataset. This method is applied in this work by aligning the quantity of all data types with that of the minority class, "outlier," significantly decreasing the training time and computational resources required. This prevents the development of biased training features, ensuring a balanced representation across signal classes. The model uses a small number of samples and achieves a decent performance.

Dataset preprocessing. After balancing, the dataset comprises 3661 samples, each initially with 72,000 data points. Because the input of a transformer encoder is in the form of three dimensions (batch size, sequence length, and feature dimension). Data points are divided into two dimensions: a sequence length of 1 and a feature dimension of 1024 per sequence. The extracted features include three statistical measures and histogram bins that represent the data's frequency distribution, providing a comprehensive view of its inherent

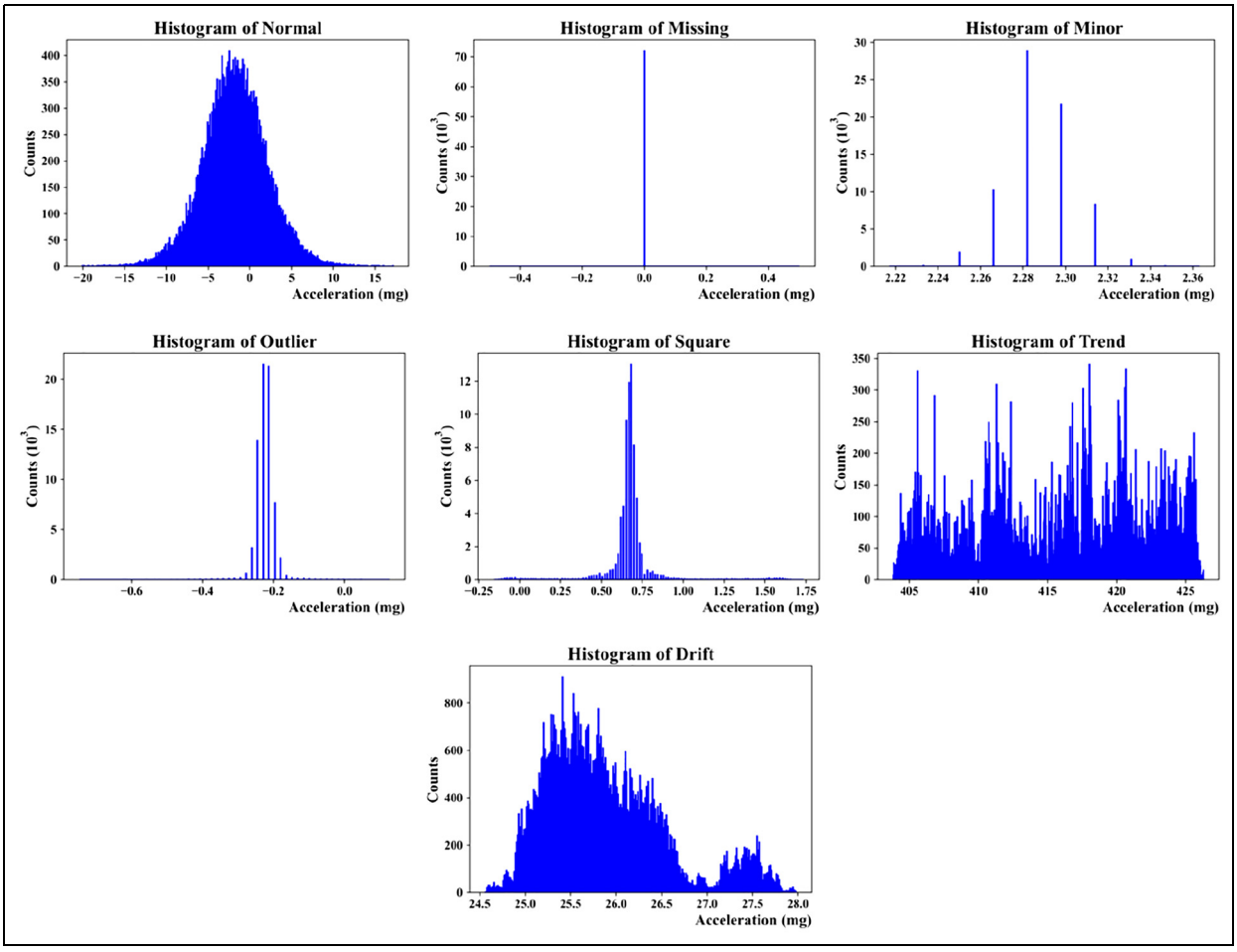


Figure 10. Histogram representation of different data types.

characteristics (see Figure 10 for histogram features). These features are processed and normalized to range of $[-1,1]$ to avoid the scale effect. They are further enhanced through a transformer encoder to improve representation and are flattened before being input into the classifier. The dataset is divided into training, validation, and testing subsets of 7:1.5:1.5. The training data are used to develop a model for seven-class anomaly classification, while the validation data help monitor and refine the model's performance. Finally, the testing data are employed to evaluate classification performance using metrics of accuracy.

Model development

The input dataset is structured into three dimensions, serving as the foundation for developing and pretraining the EECM. The EECM integrates a classifier and a transformer encoder to enhance feature extraction and representation. During training, the parameters of the EECM were fine-tuned using various hyperparameter configurations to optimize performance. The training dataset was processed in batches to adjust model parameters. Grid search is applied to select the best hyperparameter combination. This involved defining a grid of hyperparameter values, training the neural network for each combination, and evaluating performance on a validation dataset. The best-performing hyperparameter combination was selected based on validation accuracy. Early stopping was implemented to prevent overfitting by monitoring validation performance throughout the training phase. By systematically adjusting the number of layers and neurons, the configurations are identified that strike an optimal balance between model performance and computational efficiency. The iterative testing identified the optimal hyperparameters, resulting in the best model with an average test accuracy of 96.4% and a model size of 17.31 M.

Once the EECM was fully trained, its weights and predictions were frozen to guide the training of the student model DLCM through knowledge distillation.

This involved transferring the response-based knowledge from the teacher model to the student model, enabling it to mimic the teacher's behavior and decision boundaries. The DLCM was designed as a lightweight deep neural network with three layers, comprising an input layer of 1024 neurons, two hidden layers with 64 and 32 neurons, respectively, and an output layer representing seven classes. The training process was further optimized through a combined loss function integrating the distillation loss, which quantified the divergence between the teacher's soft label predictions and the student's outputs, and the classification loss, which measured the discrepancy between the student's predictions and the ground truth labels. By balancing these loss components with a tunable hyperparameter, the DLCM effectively learned to emulate the teacher's knowledge while achieving high classification precision. All experiments processed the train dataset in batches to enable systematic updates to model parameters and implemented early stopping to mitigate overfitting by monitoring validation performance throughout the training phase. Additionally, the Adam optimizer and dynamic learning rate adjustment were utilized to enhance model performance and accelerate convergence. These techniques collectively resulted in the DLCM that maintained high classification accuracy while being well-suited for deployment on resource-constrained edge devices.

Experimental results

Performance evaluation of the proposed model among different versions. This section presents the final accuracy of the model and analyzes the results running on different platforms. Figure 11 presents the confusion matrices for the DLCM, EECM, and base classification model (BCM). The horizontal axis denotes the actual labels, while the vertical axis represents the predicted labels. The diagonal elements of the matrix indicate the percentage of correctly classified instances, while the off-diagonal cells reflect the percentage of misclassifications. The classification accuracies on the test dataset are 96.36%, 96.4%, and 93.6%, respectively, highlighting the efficacy of knowledge distillation in enhancing the DLCM's performance. Figure 11(b) shows that the EECM achieved a classification accuracy exceeding 98% over all data types, except for the "outlier" category. The relatively lower accuracy for the "outlier" category may stem from the similarity in probability distributions between "minor" and "outlier" anomalies due to the low-resolution sensors used.[15] This overlap in feature characteristics increases the likelihood of misclassification between these two categories. Additionally, since "outlier" data are the least represented category, all used during dataset balancing, the possibility of incorrect labeling during the initial manual labeling process will influence the misclassification. Despite this challenge, the DLCM, after applying KD, demonstrates significant improvements in classifying "outlier" anomalies, achieving greater precision and robustness. This improvement can be attributed to the KD process, which leverages both hard labels and soft labels. The use of soft labels allows the student model to capture the interrelations among different anomaly types, enhancing its ability to generalize and replicate the teacher model's nuanced understanding of the data. When compared to the BCM, the DCLM not only enhances overall classification accuracy but also achieves better performance across all data types, demonstrating its ability to effectively replicate the EECM's capabilities and remarkable proficiency in accurately classifying anomalous data types. Figure 12 displays the training loss and validation accuracy across epochs for both the DLCM and the EECM. The results demonstrate that the DLCM successfully replicates the performance of the EECM, showcasing stability and robustness throughout the training process. Initially, the training loss decreases rapidly while the validation accuracy increases quickly. Over successive epochs, the training and validation processes synchronize closely, eventually converging to a stable state. The model exhibited robustness across different types of data and maintained stable performance throughout the training process.

Performance evaluation of the proposed model on both the PC and an IoT node. This section presents the performance evaluation of the DLCM across different platforms, including on desktop PC and an STM32-based IoT node, (the simulated environment on the PC and real edge devices on *LiftNode*), focusing on classification accuracy, inference time, and resource utilization. The program is tested using the test dataset of 550 samples with feature dimension 1024 across 7 classes, including 64 "normal," 86 "missing," 89 "outlier," 73 "minor," 67 "square," 89 "trend," and 82 "drift" instances. Figure 13 presents the prediction confusion matrix of edge computing on *LiftNodes*, where the horizontal axis represents the actual labels, and the vertical axis indicates the predicted labels. The diagonal elements of the matrix correspond to the number of correctly classified instances, providing insight into the model's performance. The edge device processes one sample at a time, and raw data segments (e.g., 72,000 points) are buffered temporarily to compute features. 1024-dimensional features represented in Float32 format are extracted and input to the model, consuming 4 KB of memory (1024 dimensions $\times$ 1 sequence $\times$ 4 bytes/1024 KB). The RAM usage of the histogram and

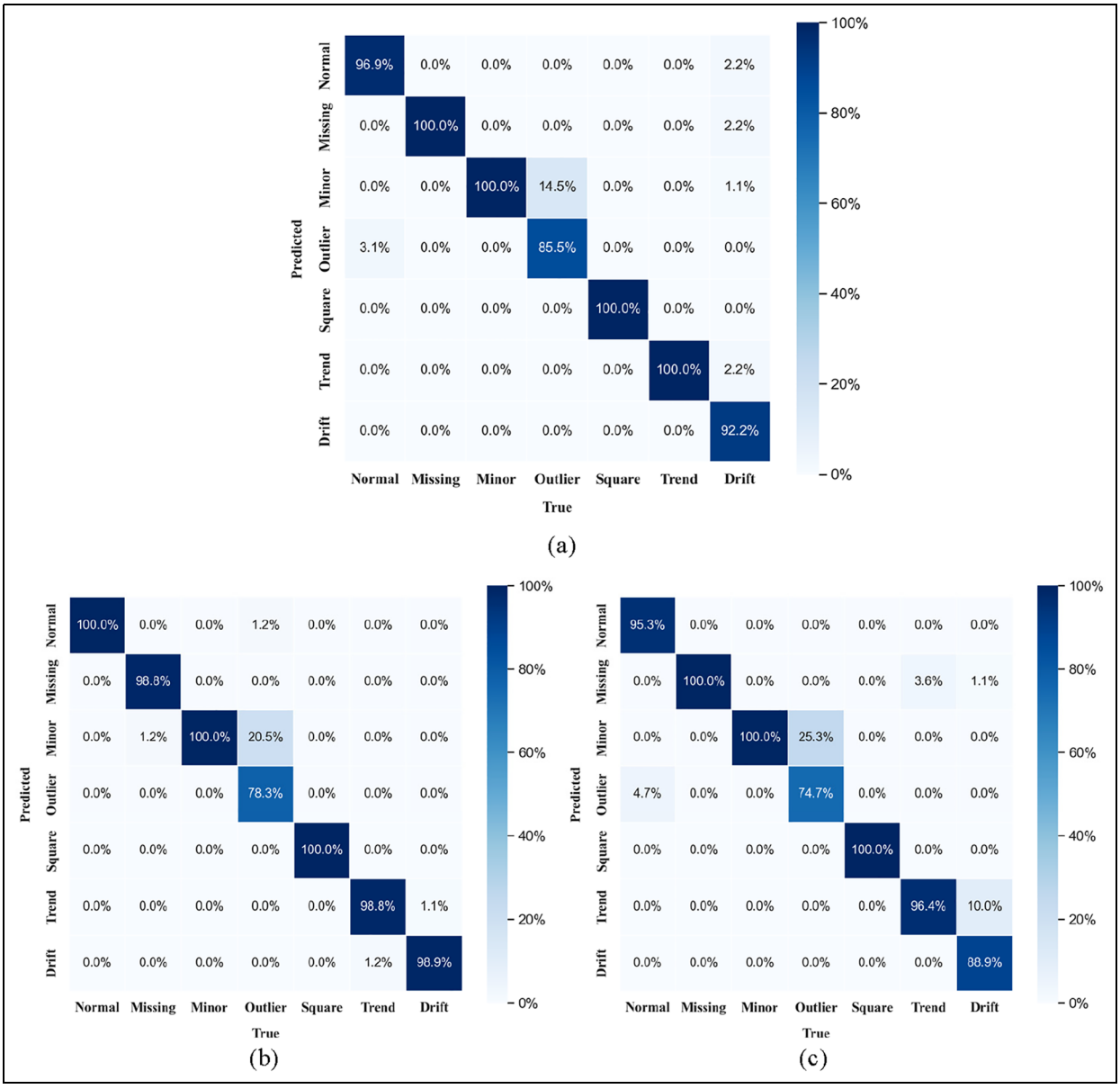


Figure 11. Confusion matrix of (a) DLCM, (b) EECM and (c) BCM.
DLCM: distilled lightweight classification model; EECM: encoder enhanced classification model; BCM: base classification model.

statistical feature extraction module is approximately 2.5 KB and has been added to the total memory consumption budget. Table 1 summarizes the comparison of the DLCM's accuracy, inference time, and resource utilization across the platforms. The Python model represents the classifier code of the DLCM written in Python running through VScode on PC, achieving a prediction accuracy of 96.37% with an inference time of 0.61 ms for the entire test dataset. The x86 C-model, compiled and executed using X-CUBE-AI, achieved a comparable accuracy of 96.36% with a slightly longer inference time of 3.55 s for 550 samples, translating to approximately 0.07 ms per sample refers to the classifier implemented in C codes using X-CUBE-AI, running predictions in a simulation edge deceive environment. When deployed on the STM32-based IoT node, the C-model maintained an accuracy of 96.36%, showcasing the model's robustness and consistency across different hardware architectures. The inference time of overall 550 samples was 339.35 s on the MCU with an average inference time of 0.62 s for each sample. This increase in inference time is attributed to the computational constraints of the MCU compared to the desktop environment. In terms of

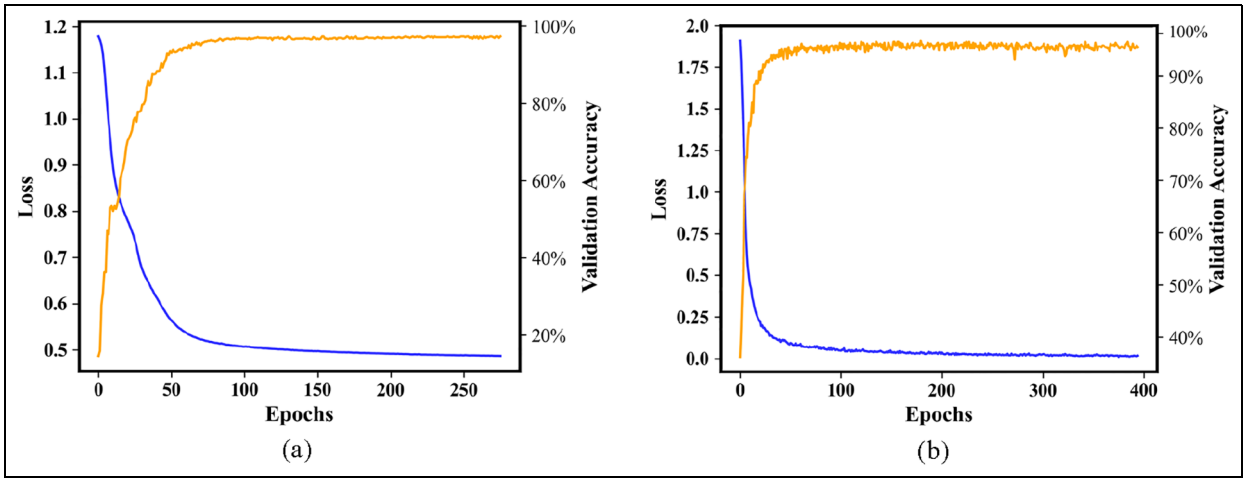


Figure 12. Training loss and validation accuracy per epoch for (a) DLCM and (b) EECM.
DLCM: distilled lightweight classification model; EECM: encoder enhanced classification model.

resource usage, the flash memory usage of the board on *LiftNode* is 296.9 KB out of the available 1 MB, and the RAM usage is 4.25 KB out of the total 340 KB. These results indicate that the DLCM, with its lightweight architecture, fits well within the memory limitations of the MCU-based resource-constrained IoT node. The test batch size was set to 64 samples for desktop environments, while the STM32 board processed one sample at a time due to its limited processing power. Overall, these findings demonstrate the DLCM's adaptability and efficiency for edge intelligence applications. By achieving high classification accuracy and conforming to the hardware constraints of IoT nodes, the model proves its viability for deployment in real-world, resource-constrained environments.

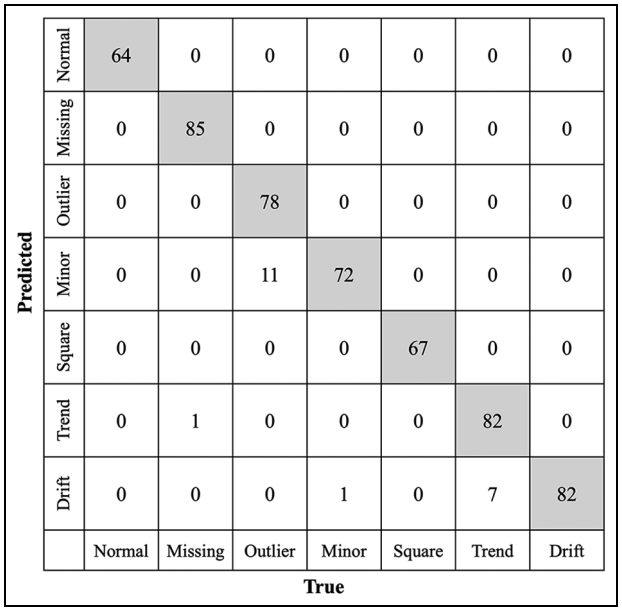


Figure 13. Predict confusion matrix on target.

Discussions

The comparative analysis of various deep learning models, as shown in Figure 14, highlights the trade-off between model size and classification accuracy. GoogLeNet, as used in Mondal et al.,[26] achieved high accuracy but has complex architecture and large model size. Although hybrid compression techniques were applied including pruning approaches, depthwise separable convolution (DSC), and quantization-aware training (QAT), the smallest model size reported 0.7 MB still exceeded that of our proposed method. Furthermore, the accuracy declines with compression.

Table 1. Accuracy and time of the DLCM running on different platforms.

Model type	Accuracy	Params	Inference time	Inference time/sample (ms)	Platform
Python model	96.37%	69,711	0.61 s/550samples	0.011	Desktop
x86 c-model	96.36%	68,112	3.554 s/550samples	0.065	Desktop
stm32 c-model	96.36%	68,112	339.35 s/550samples	6.17	Stm32 board

DLCM: distilled lightweight classification model.

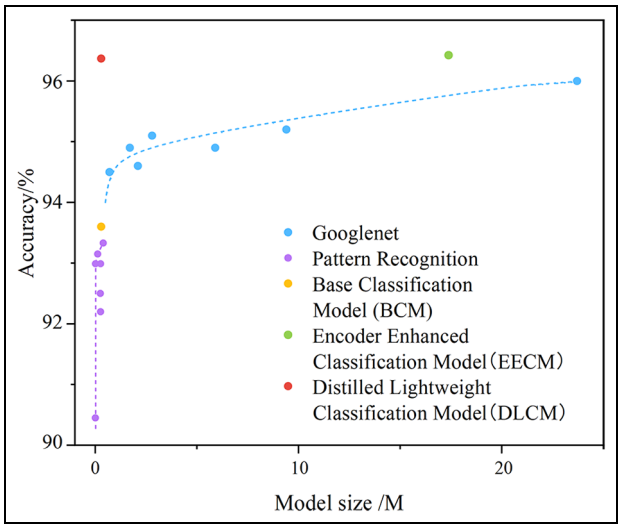


Figure 14. The accuracy versus model size among different models.

The pattern recognition model by Chou et al.[15] featured a small model size but achieved only moderate accuracy of 93.3%, reflecting a limitation in their ability to effectively classify complex data anomalies. The EECM achieves superior accuracy due to its advanced architecture and feature extraction capabilities, but its model size renders it unsuitable for edge applications. In contrast, the BCM demonstrates significant computational efficiency with a smaller size, yet at the expense of reduced accuracy. Notably, the DLCM, after applying knowledge distillation strikes an optimal balance, achieving a classification accuracy comparable to the EECM while maintaining a compact size. This improvement underscores the efficacy of the KD process in transferring critical information from the teacher model to a computationally efficient student model. This method leverages both hard labels and soft labels to train the DLCM, enabling it to mimic the teacher model's decision boundaries and interclass relationships. The significance of KD lies in its ability to retain high classification accuracy while achieving substantial reductions in model size and computational complexity, making it a compelling strategy for edge intelligence.

Additionally, this study retained the one-dimensional nature of the raw time-series data without converting it into images or other three-dimensional features, avoiding the additional computational burden and storage requirements associated with higher-dimensional representations. The streamlined approach to feature extraction, utilizing statistical and histogram features from one-dimensional data, ensures efficient classification with a smaller model size and mitigates risks of overfitting, particularly in scenarios with limited or imbalanced datasets. Each input sample, represented in Float32 format, occupies 4 KB of memory (1024 dimensions $\times$ 1 sequence $\times$ 4 bytes/1024 KB). In contrast, GoogLeNet[26] leverages time–frequency image representations, which expand the data from one dimension to three dimensions. While this method captures richer features and often yields promising performance, it significantly increases data complexity and storage requirements. A single image in this representation, with a resolution of $H = 224 \times W = 224$ and three-color channels, occupies approximately 147 KB of memory ($224 \times 224 \times 3$ channels $\times$ 4 bytes/1024 KB). Such requirements pose challenges for large-scale datasets and edge-computing applications where memory and storage resources are limited. By contrast, the proposed DLCM achieves a favorable balance between simplicity, efficiency, and accuracy, demonstrating its practicality for deployment in resource-constrained environments, where both computational and storage constraints are critical. These results demonstrate the effectiveness of KD in training lightweight neural networks for data anomaly classification tasks.

Conclusion

This study developed an efficient edge intelligence strategy using knowledge distillation for onboard data anomaly classification for wireless SHM systems. It first trained an EECM that adopted domain knowledge and a transformer encoder to effectively extract features, making the subsequent task of classification easy. After that, a compact model DLCM is trained by transferring response-based knowledge from teacher models and guided by distillation loss and classification loss. As a result, it not only achieves high accuracy for data anomaly classification but also simplifies the model complexity significantly, suitable for edge computing. Furthermore, this study built a comprehensive edge computing framework for deploying machine learning models on the computing board of a low-cost IoT node, which has constraints in battery power, clock speed, and computational memory. The proposed study offers the advantages of lower cost and higher energy efficiency compared to single-board-computers-based edge computing solutions (e.g., Raspberry Pi and Jetson in most state-of-the-art solutions), making it ideal for large-scale cost-effective structural health monitoring applications. Utilizing the proposed efficient edge intelligence techniques, raw data can be collected and processed locally on edge devices, before being transmitted to a central station for wireless structural health monitoring systems. Future work will investigate the scalability of the proposed method

across diverse scenarios and assess interoperability with heterogeneous sensor networks in the SHM. Long-term performance evaluations will be conducted to examine reliability, robustness, and lifecycle efficiency in real-world environments. Additionally, onboard sensor fault diagnosis mechanisms will be developed to trace data anomalies to their root causes and develop recovery strategies.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This research was supported by The Ministry of Education Tier 1 Grants, Singapore (No. RG121/21, RS04/21, and RG136/22), and the start-up grant at Nanyang Technological University, Singapore (03INS001210C120).

Ethical considerations

Not applicable.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Data availability

Data will be made available on request.

ORCID iDs

Xiao Yu https://orcid.org/0009-0007-8294-5671
Xinhao He https://orcid.org/0000-0002-7162-6719
Yuguang Fu https://orcid.org/0000-0001-7125-0961

References

1. Scuro C, Lamonaca F, Porzio S, et al. Internet of Things (IoT) for masonry structural health monitoring (SHM): overview and examples of innovative systems. *Construc Build Mater* 2021; 290: 123092.
2. Poornima IGA and Paramasivan B. Anomaly detection in wireless sensor network using machine learning algorithm. *Comput Commun* 2020; 151: 331-337.
3. Mao J, Su X, Wang H, et al. Automated Bayesian operational modal analysis of the long-span bridge using machine-learning algorithms. *Eng Struct* 2023; 289: 116336.
4. Yu X, Fu Y, Li J, et al. Recent advances in wireless sensor networks for structural health monitoring of civil infrastructure. *J Infrastruct Intell Res* 2023; 3: 100066.
5. Su X, Mao J, Wang H, et al. Deep learning-based automated identification on vortex-induced vibration of long suspenders for the suspension bridge. *Mech Syst Signal Process* 2025; 224: 112070.
6. Zhou G-D and Yi T-H Recent developments on wireless sensor networks technology for bridge health monitoring. *Math Probl Eng* 2013; 1: 947867.
7. Rassam MA, Zainal A and Maarof MA. Advancements of data anomaly detection research in wireless sensor networks: a survey and open issues. *Sensors* 2013; 13: 10087–10122.
8. Yi T-H, Huang H-B and Li H-N Development of sensor validation methodologies for structural health monitoring: a comprehensive review. *Measurement* 2017; 109: 200–214.
9. Bao Y, Li H, Sun X, et al. Compressive sampling–based data loss recovery for wireless sensor networks used in civil structural health monitoring. *Struct Health Monit* 2013; 12: 78–95.
10. Mo L, Li J and Wang, G. et al. Passive diagnosis for WSNs using time domain features of sensing data. *Int J Distrib Sens Netw* 2015; 11: 590430.
11. Samudra S, Barbosh M and Sadhu A. Machine learning-assisted improved anomaly detection for structural health monitoring. *Sensors* 2023; 23: 3365.
12. Bao Y, Li J, Nagayama T, et al. The 1st international project competition for structural health monitoring (IPC-SHM, 2020): a summary and benchmark problem. *Struct Health Monit* 2021; 20: 2229–2239.
13. Fu Y, Peng C, Gomez F, et al. Sensor fault management techniques for wireless smart sensor networks in structural health monitoring. *Struct Control Health Monit* 2019; 26: e2362.
14. Mao J, Su X, Gui G, et al. ResNet transfer learning for accurate and efficient anomaly detection of bridge vibration data. *Smart Struct Syst* 2024; 34(6): 415.
15. Chou J-Y, Fu Y, Huang S-K, et al. SHM data anomaly classification using machine learning strategies: a comparative study. *Smart Struct Syst* 2022; 29: 77–91.
16. Rajasegarar S, Leckie C and Palaniswami M. Anomaly detection in wireless sensor networks. *IEEE Wireless Commun* 2008; 15: 34–40.
17. Al-amri R, Murugesan RK, Man M, et al. A review of machine learning and deep learning techniques for anomaly detection in IoT data. *Appl Sci* 2021; 11: 5320.
18. Cha Y-J, Ali R, Lewis J, et al. Deep learning-based structural health monitoring. *Autom Constr* 2024; 161: 105328.
19. Mao J, Wang H and Spencer BF Jr. Toward data anomaly detection for automated structural health monitoring: Exploiting generative adversarial nets and autoencoders. *Struct Health Monit* 2021; 20: 1609–1626.
20. Ni F, Zhang J and Noori MN. Deep learning for data anomaly detection and data compression of a long-span suspension bridge. *Comput Aid Civ Infrastruct Eng* 2020; 35: 685–700.

21. Arul M and Kareem A. Data anomaly detection for structural health monitoring of bridges using shapelet transform. arXiv preprint, arXiv:2009.00470, 2020.
22. Zhang Y, Tang Z and Yang R. Data anomaly detection for structural health monitoring by multi-view representation based on local binary patterns. *Measurement* 2022; 202: 111804.
23. Flah M, Nunez I, Ben Chaabene W, et al. Machine learning algorithms in civil structural health monitoring: a systematic review. *Arch Comput Methods Eng* 2021; 28: 2621–2643.
24. Zhang H, Lin J, Hua J, et al. Data anomaly detection for bridge SHM based on CNN combined with statistic features. *J Nondestruct Eval* 2022; 41: 28.
25. Tang Z, Chen Z, Bao Y., et al. Convolutional neural network-based data anomaly detection method using multiple information for structural health monitoring. *Struct Control Health Monit* 2019; 26: e2296.
26. Mondal TG, Chou J-Y, Fu Y, et al. A hybrid deep neural network compression approach enabling edge intelligence for data anomaly detection in smart structural health monitoring systems. *Smart Struct Syst* 2023; 32: 179–193.
27. Luo T and Nagarajan SG. Distributed anomaly detection using autoencoder neural networks in WSN for IoT. In: *2018 IEEE international conference on communications (ICC)*, Kansas City, MO, pp. 1–6. IEEE.
28. Ding Y, Yang K, Jiang H, et al. Data abnormal detection using bidirectional long-short neural network combined with artificial experience. *Smart Struct Syst Int J* 2022; 29(1): 117–127.
29. Liu G, Niu Y, Zhao W, et al. Data anomaly detection for structural health monitoring using a combination network of GANomaly and CNN. *Smart Struct Syst* 2022; 29(1): 53–62.
30. Bao Y, Tang Z, Li H, et al. Computer vision and deep learning–based data anomaly detection method for structural health monitoring. *Struct Health Monit* 2019; 18: 401–421.
31. Fu Y, Hoang T, Mechitov K, et al. xShake: intelligent wireless system for cost-effective real-time seismic monitoring of civil infrastructure. *Smart Struct Syst* 2021; 28(4): 483–497.
32. Rajasegarar S, Leckie C, Palaniswami M., et al Quarter sphere based distributed anomaly detection in wireless sensor networks. In: *2007 IEEE international conference on communications*, Glasgow, pp. 3864–3869. IEEE.
33. Zhang Y, Hamm NA, Meratnia N, et al. Statistics-based outlier detection for wireless sensor networks. *Int J Geographical Inform Sci* 2012; 26(8): 1373–1392.
34. Branch JW, Giannella C, Szymanski B, et al. In-network outlier detection in wireless sensor networks. *Knowledge Inform Syst* 2013; 34: 23–54.
35. Rassam MA, Zainal A and Maarof MA. An efficient distributed anomaly detection model for wireless sensor networks. *AASRI Proc* 2013; 5: 9–14.
36. Wang X, Wu W, Du Y, et al. Wireless IoT monitoring system in Hong Kong–Zhuhai–Macao bridge and edge computing for anomaly detection. *IEEE Internet Things J* 2023; 11(3): 4763–4774.
37. Geck CC, Thamer A-Z, Elmoursi M, et al. AIoT-enabled decentralized sensor fault diagnosis for structural health monitoring. In: *11th European workshop on structural health monitoring*, 2024.
38. Liu G, Li L, Zhang L, et al. Sensor faults classification for SHM systems using deep learning-based method with Tsfresh features. *Smart Mater Struct* 2020; 29: 075005.
39. Zhao R, Yan R, Chen Z, et al. Deep learning and its applications to machine health monitoring. *Mech Syst Signal Process* 2019; 115: 213–237.
40. Kim S-Y and Mukhiddinov M. Data anomaly detection for structural health monitoring based on a convolutional neural network. *Sensors* 2023; 23: 8525.
41. Jan SU, Lee Y-D, Shin J, et al. Sensor fault classification based on support vector machine and statistical time-domain features. *IEEE Access* 2017; 5: 8682–8690.
42. Li J, Chen W and Fan G. Structural health monitoring data anomaly detection by transformer enhanced densely connected neural networks. *Smart Struct Syst* 2022; 30: 613–626.
43. Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need. In: *Advances in neural information processing systems*, 2017, Los angeles, CA, p. 30.
44. Hinton G, Vinyals O and Dean J. Distilling the knowledge in a neural network. arXiv preprint; arXiv:1503.02531, 2015.
45. Gou J, Yu B, Maybank SJ, et al. Knowledge distillation: a survey. *Int J Comput Vision* 2021; 129(6): 1789–1819.
46. Fu Y, Zhu Y, Hoang T, et al. xImpact: intelligent wireless system for cost-effective rapid condition assessment of bridges under impacts. *Sensors* 2022; 22(15): 5701.
47. Al-Qamash A, Soliman I, Abulibdeh R, et al. Cloud, fog, and edge computing: a software engineering perspective. In: *2018 International conference on computer and applications (ICCA)*, Beirut, Lebanon, pp. 276–284. IEEE.
48. Cao K, Liu Y, Meng G, et al. An overview on edge computing research. *IEEE Access* 2020; 8: 85714–85728.
49. Park J-W, Sim S-H, Jung H-J, et al. Development of a wireless displacement measurement system using acceleration responses. *Sensors* 2013; 13: 8377–8392.
50. Lynch JP, Wang Y, Loh KJ, et al. Performance monitoring of the Geumdang bridge using a dense network of high-resolution wireless sensors. *Smart Mater Struct* 2006; 15: 1561.